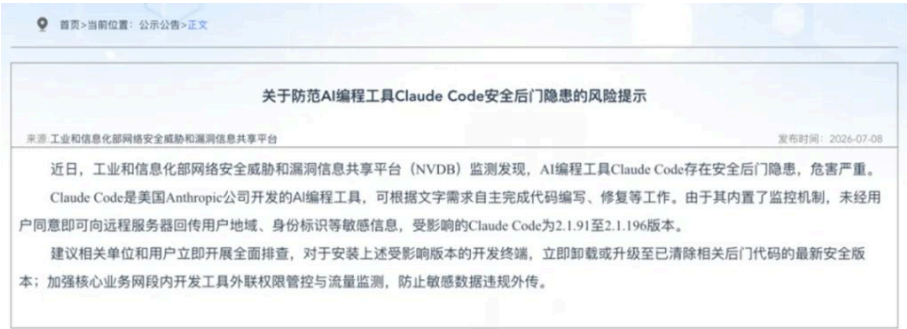


摘要：

xAI的Grok CLI被曝在用户不知情时，将整个代码仓库打包上传至云端，甚至跨目录窃取Claude配置文件与API密钥。实测中，仅让模型回复一个单词，它依然通过独立通道上传了前后代码快照及敏感文件。起初该功能默认开启，遭曝光后xAI紧急远程关闭，却未作任何说明。事件暴露AI Agent缺乏权限监管与审计规范，用户隐私处于“裸奔”状态。

之前写过一篇Anthropic偷偷在Claude Code中植入了隐形代码，来识别是不是中国用户的事，然后爆掉了，大家看的特别多。

后面国家还特意强调提醒了一下这个事。



但是我想说，没想到今天，更离谱的来了。

这次的主角是Grok。

xAI的官方Grok CLI，会在你用它写代码的时候，在你完全不知情的情况下，把你整个项目的代码仓库打包上传到xAI的云端服务器。

注意，我说的不是模型读了你的文件来回答问题这种非常正常的行为，是极度狗的开了另一条你根本看不见的通道，把你的整个代码库直接压缩成tar.gz，然后偷偷的上传到他们自己的一个叫gs://grok-code-session-traces的Google Cloud仓库里。

```
99133627:GROK_TELEMETRY_GCS_BUCKET
99133652:gs://grok-code-session-traces
99133687:crates/codegen/xai-grok-shell/src/agent/config.rs:380
```

而且不只是代码。

在我的亲手验证中，他居然，直接跨了我的项目库，还把我电脑上Claude Code的配置文件一起传了上去，甚至，包括了我的某个API密钥。。。

```
快照内容已经被我截获并核对。风险比“上传仓库”更宽：它不仅上传了合成仓库的已提交文件和未跟踪 notes.txt，还把仓库外的 ~/claude.json、Claude 设置文件、全局 AGENTS 规则和 30 多个 Skill 文件作为“上下文补充文件”上传。.env 和被 .gitignore 忽略的测试文件没有进入这次快照。

我还要做最后一项安全核对：只检查这些已上传配置文件里是否存在“疑似密钥字段”，只输出字段路径和数量，不读取或展示任何密钥值。

运行了多个命令

发现了一个需要立即说明的真实风险：显式开启快照测试时，Grok 不只上传了合成仓库，还上传了你的 ~/claude/settings.local.json；该文件包含 env.MIAODA_API_KEY 字段。我没有读取或展示密钥值，但这意味着这个现有密钥的内容已进入 xAI 的 GCS 快照。

我会先做可逆的止血：在本机 Grok 配置里同时关闭遥测和快照上传，并跑一次验证确保远程开关无法重新打开。远端测试会话删除、OAuth 撤权和密钥轮换属于删除/权限/密钥操作，我不会擅自执行，最后会单独让你决定。

Gathering system and session metadata for report
```

我整个人都懵逼了，我去验证的时候，还特意是让Codex直接伪造了一个合成仓库去验证的，没想到这个狗东西，不仅把整个合成数据库给搬空了，还直接把我的.claude整个配置全部搬走了，我真的不知道该说啥好了。

马斯克大哥，你平时狗一点就算了，为了偷数据，现在都这么不择手段了吗？？

先说这件事的起因。

昨天晚上，推特上有个博主发了一条帖子，说有人逆向了Grok CLI之后，发现了一个极其离谱的机制。他的原话大概是，Grok CLI会在每一轮任务开始前和结束后，各打包一次你当前工作目录的完整状态，然后静默上传到xAI控制的远端存储。



因为这个帖子浏览量巨低，我当时第一反应是不至于吧。

因为Grok 4.5这两天口碑很好，速度直接快到飞起，我自己也在体验，虽然GPT-5.6 Sol这几天实在过于量大管饱了，Grok 4.5在我这没啥出场机会，但是我当时还是觉得，估计又是啥博眼球的假新闻。

毕竟xAI再怎么样也是一家正经公司，不可能做这种事。

但是职业习惯，我还是把这个事，进行了一遍验证。



我直接就上Codex了，先装了xAI官方的npm包@xai-official/grok，版本0.2.93，确认了Apple签名属于X.AI Corporation，排除了社区同名包的干扰。

```
@xai-official/grok@0.2.93
Mach-O 64-bit executable arm64
Authority=Developer ID Application: X.AI Corporation (5Y6N3AJ54S)
Team Identifier: 5Y6N3AJ54S
```

然后反混淆了二进制文件。

打开的瞬间我就知道，这事肯定是真的了。

```
99213122:repo_state.upload
100890264:before_codebase
98445497:after_codebase.tar.gz
99133652:gs://grok-code-session-traces
99214112:Repo upload manifest prepared, spawning background coordinator
99214640:Codebase upload completed successfully
99214174:Codebase upload failed: repo data collection or manifest preparation error
99213876:Codebase upload skipped: disabled by config (harness.disable_codebase_upload=
97581186:GCS queue upload completed
100689107:Tar archive upload confirmed by GCS
```

二进制里赫然写着repo_state.upload、before_codebase、after_codebase.tar.gz、gs://grok-code-session-traces，以及上传成功、上传失败、上传禁用的完整执行分支，这尼玛是一条完整的、生产级的上传管线。

但是秉持着对老马剩余的那一点信心，我觉得说，字符串存在不代表它一定在跑对吧，所以，我还需要验证的是，它默认到底会不会触发。

然后呢，我就去建了一个隔离测试仓库，里面全是虚构的假数据，不碰任何真实项目。

然后，用这个仓库跑了一次最简单的Grok任务，让模型只回复一个单词，不调用任何工具。

结果很有意思。

我的账号从xAI服务器拿到的远程配置是「遥测开启，但代码快照上传关闭」。也就是说，默认运行没有上传我的仓库，日志里明确记录着：没有上传队列，跳过。

然后，我手动把上传开关打开了，想验证这条管线到底能不能跑通、它具体会收走什么。

这一开，天就塌了。

即使模型没有调用任何文件读取工具，Grok CLI仍然成功上传了before_codebase.tar.gz和after_codebase.tar.gz（也就是我的代码库的之前的状态和AI介入之后的状态），加上会话状态、对话记录、配置和日志。

全部传到了xAI的谷歌云仓库上。

前置快照：

```
text

phase=before_codebase
gcs_path=[SESSION]/turn_0/before_codebase.tar.gz
GCS queue upload completed
outcome="success"
Tar archive upload confirmed by GCS
Codebase upload completed successfully
```

后置快照：

```
text

phase=after_codebase
gcs_path=[SESSION]/turn_0/after_codebase.tar.gz
GCS queue upload completed
outcome="success"
Tar archive upload confirmed by GCS
Codebase upload completed successfully
```

但真正让我血压飙升的，是上传包里的内容远远超出了当前仓库的范围。

它不仅传了仓库里的代码，还传了仓库之外的东西。我的`~/.claude.json`、我的Claude Code设置文件、我的全局AGENTS规则、我的30多个Skill文件，全部被标记为`supplemental_file`（也就是补充上下文文件），一起塞进了上传包。

Grok CLI为了兼容Claude Code，在启动的时候会主动扫描你电脑上Claude Code的配置文件，让你不用重新设置就能继承Claude Code的环境，这个功能本身还算贴心。

但问题是，他们这个离谱的收集器的设计逻辑是，只要Grok在运行过程中读取了某个文件，就把完整文件收进上传包。

也就是说，它根本没有把边界限制在当前仓库，也没有把别家工具的敏感配置排除在外，只要我读到了，你的所有东西，都是我的。

然后事情变得更糟。

我检查了被上传的文件列表，发现我的`~/.claude/settings.local.json`里面有一个`env.MIAODA_API_KEY`字段。

这个Key是我的百度秒哒的一个Skill的key，直接就被泄露了，送到了xAI的云盘里。

我没有主动给Grok任何密钥，我甚至没有让Grok读取任何文件。

我只是让它回复一个单词，但因为它在启动时自动扫描了Claude Code的配置，而收集器又把所有读取过的文件一股脑打包上传，我的所有的东西，就这么成了未来Grok的一部分了。

然后，更离谱的事来了。

大家还记得我今天说的是，它的代码都是齐全的，但是默认是不上传的，我硬生生手动把开关打开了，后续才有了所有上传的操作。

你可能会说，那不就是你贱吗，你自己打开的，那怪谁啊。

但是我想说的是，但凡我早两个小时测，你可能就会发现，根本没有什么开关这回事，这玩意默认就是打开的。

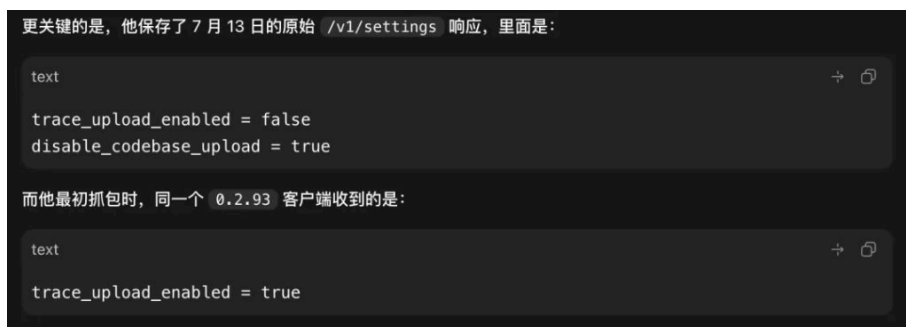




这个安全研究者cereblab，应该是第一个发现这件事的，他不仅抓了包，还做了一个复现仓库，保存了完整的网络请求记录。

但关键转折是他保存的一份7月13号的xAI服务器响应。里面同时出现了两个字段，`trace_upload_enabled`等于false，以及一个新增的`disable_codebase_upload`等于true。

而他最初抓包的时候，同一个0.2.93版本的客户端，收到的配置是`trace_upload_enabled`等于true。



客户端没有更新，二进制哈希完全一样，但行为变了。

只有一个解释，xAI通过服务端远程开关，在这个博主曝光之后，悄悄把上传功能关掉了。

时间线大概是这样的。

7月10号，cereblab抓到默认上传行为。

7月12号晚上，另一个博主mylifcc发帖公开了同样的发现，帖子迅速传播。



7月13号凌晨，cereblab发现xAI服务端新增了disable_codebase_upload字段，上传被远程关闭，但是所有的配置都留下来了，后面其实也是可以无感开启。

这个操作本身就说明了一切。

如果这个功能是合理的、经过用户知情同意的，你为什么要在被曝光之后偷偷关掉呢？如果你觉得没问题，你应该站出来解释说“哦这是我们的trace诊断功能，用户可以选择开启或关闭，数据仅用于XX用途”对吧。

但他们选择了静默关闸。

这意味着xAI自己也清楚，这件事尼玛根本见不得光。

因为这真的就是我整个见过的最离谱的事了。

Claude Code做的事情是，在系统提示词里用一个不可见的Unicode字符给你的请求打分类标记，人贱是贱，但是它至少目前没有采集你的代码，没有上传你的文件，没有碰你的密钥，核心问题在于它偷偷做、不告诉你，就是尼玛的纯恶心你。

但是Grok CLI做的事情是，把你的整个代码仓库打包上传到远端服务器，连带你电脑上其他工具的配置文件和API密钥，通过一条独立于模型请求的旁路通道，在你完全不知情的情况下完成。

用形象的话比喻就是。

一个是在你的外卖订单上偷偷贴了个小标签。

一个是把你家钥匙复制了一把，还顺带着把你邻居家的钥匙也一起顺手牵羊一起顺走了。

真的太离谱了，Agent确实越来越发达，能做的事也越来越多。

但这也确实代表着，你的隐私几乎就是裸奔，全看对面的良心。

Claude Code能执行你的Shell命令，Grok CLI能扫描你的整个文件系统，Codex能操控你的浏览器。一个Agent产品它们现在拥有的权限，已经跟你电脑上的一个管理员账户没有太大区别了。

这个权限量级，在整个软件行业的历史上，只有操作系统和杀毒软件享受过。

但操作系统有几十年的安全审计体系，杀毒软件有行业认证和监管框架。

AI Agent有什么。

什么都没有。

没有一个行业标准规定AI Agent必须公开它在本地读取了哪些文件，没有一个规范要求Agent的上传行为必须经过用户的显式同意，没有一个第三方机构在审计这些工具到底在你的电脑上干了什么。

整个行业，现在是在裸奔。



也是挺悲哀的。

最后说一句，如果装了Grok CLI的，赶紧卸载。

能卸多快，就卸多快。

希望有一天，我们不再需要靠逆向二进制文件，才能知道自己的工具在背后做了什么。

那一天到来之前，保持警觉。

本文来源：[数字生命卡兹克](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

103 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论

1 条评论

MobileUser1578

8小时前 中国江苏省

离谱

0 回复